

code can be easily modified to handle more GPIOs. Code accepts GPIO number from user and controls it.

Create another file with command 'sudo nano led.c' for the code of the application program and copy below code in it.

```
----- led.c -----
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <errno.h>
#include <unistd.h>
#include <fcntl.h>

int gpio_export(unsigned int );
int gpio_unexport(unsigned int);
int gpio_set_dir(unsigned int ,
unsigned int );
int gpio_set_value(unsigned int ,
unsigned int );

#define SYSFS_GPIO_DIR "/sys/class/
gpio"
#define MAX_BUF 64

int main()
{
    int x;
    printf("select the gpio ");
    scanf("%d",&x); // accept GPIO number
    from user

    gpio_export(x); // export that gpio
    to user space
    gpio_set_dir(x,1); // set
        direction of gpio as 1= output

    while(1) //loop for blinking the led
    {
        gpio_set_value(x,1); // glowing led
            connected to gpio x.
        printf("LED ON\n");
        sleep(1); // delay of 1 second
        gpio_set_value(x,0); // led off
        printf("LED OFF\n");
        sleep(1);
    }
    // making GPIO free for other kernel
    modules if loop is finite
    gpio_unexport(x);
    return 0;
}

/*****Export function*****/
```

```
int gpio_export(unsigned int gpio)
{
    int fd, len;
    char buf[MAX_BUF];

    //opening EXPORT file as Write Only
    fd = open(SYSFS_GPIO_DIR "/export",
O_WRONLY);
    if (fd < 0) // error in opening file
    {
        perror("gpio/export");
        return fd;
    }

    len = snprintf(buf, sizeof(buf),
"%d", gpio);
    write(fd, buf, len);
    close(fd);

    return 0;
}

/***** gpio_unexport *****/
int gpio_unexport(unsigned int gpio)
{
    int fd, len;
    char buf[MAX_BUF];

    fd = open(SYSFS_GPIO_DIR "/unexport",
O_WRONLY);
    if (fd < 0)
    {
        perror("gpio/unexport");
        return fd;
    }

    len = snprintf(buf, sizeof(buf),
"%d", gpio);
    write(fd, buf, len);
    close(fd);
    return 0;
}

/***** gpio_set_direction *****/
int gpio_set_dir(unsigned int gpio,
unsigned int dir)
{
    int fd, len;
    char buf[MAX_BUF];

    len = snprintf(buf, sizeof(buf), SYSFS_
GPIO_DIR "/gpio%d/direction", gpio);

    fd = open(buf, O_WRONLY);
    if (fd < 0)
```

```
{
    perror("gpio/direction");
    return fd;
}

if (dir) // 1=output and 0=input
write(fd, "out", 4);
else
write(fd, "in", 3);

close(fd);
return 0;
}

/***** gpio_set_value *****/
int gpio_set_value(unsigned int gpio,
unsigned int val)
{
    int fd;
    char buf[MAX_BUF];

    len = snprintf(buf, sizeof(buf),
SYSFS_GPIO_DIR "/gpio%d/value",
gpio);

    fd = open(buf, O_WRONLY);
    if (fd < 0)
    {
        perror("gpio/set-value");
        return fd;
    }

    if (val)
    write(fd, "1", 2);
    else
    write(fd, "0", 2);

    close(fd);
    return 0;
}
```

After copying the code, use 'ctrl+o' to save and 'ctrl+x' to exit. To compile the code, use in-built gcc compiler in Linux and execute the code that will use SYSFS interface to get the LED blinking connected at a GPIO.

To compile the code:

```
$ gcc -o sysled led.c
```

To execute the code:

```
$ sudo ./sysled
```

where sysled is the output filename. ●

Onkar Ashok Mate is pursuing M.Tech in Electronics from Walchand College of Engineering, Sangli, Maharashtra and V.B. Dharmadikari is an associate professor at Department of Electronics, Walchand College of Engineering, Sangli, Maharashtra